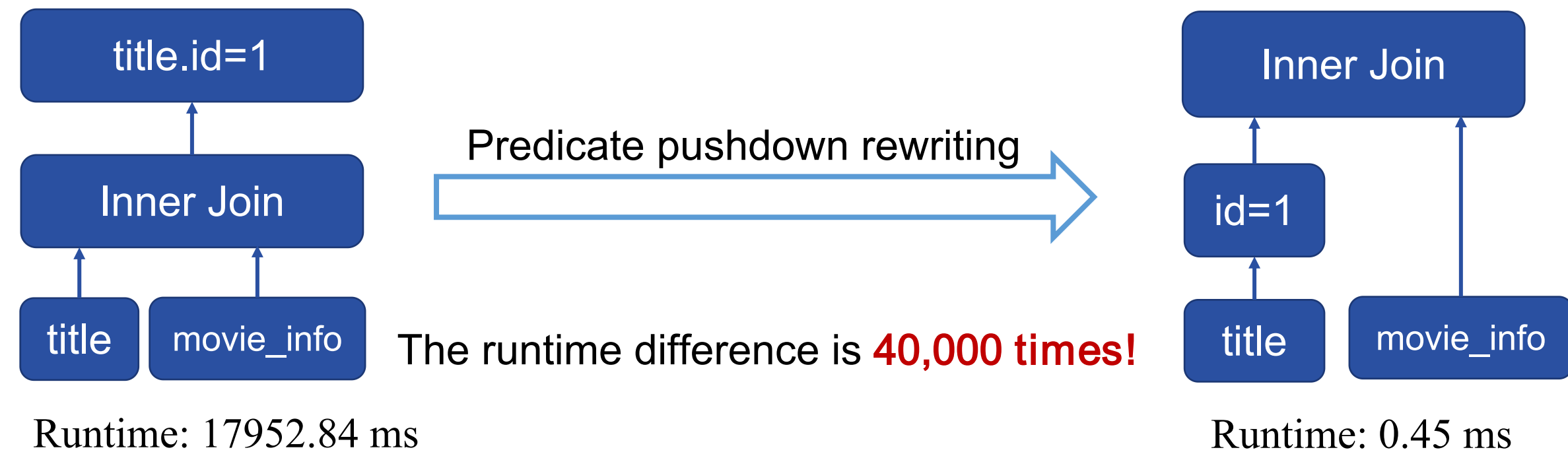


Efficient Query Rewrite Rule Discovery via Standardized Enumeration and Learning-to-Rank

Yuan Zhang¹ Yuxing Chen^{†2} Yuekun Yu¹ Jinbin Huang¹ Rui Mao¹ Anqun Pan^{†2} Lixiong Zheng^{†2} Jianbin Qin¹
¹ Shenzhen Institute of Computing Science, Shenzhen University ²Tencent Inc.

A. Introduction

- **Background:** Query rewriting is a cornerstone of query optimization. While manual rule creation is slow, automated discovery tools (e.g., WeTune) have emerged.



- **Motivation:** Modern production systems require complex rules (5-10 nodes), but existing tools face a "complexity wall" due to exponential search spaces.
- **Core Goal:** To build a scalable system that discovers high-impact, complex rewrite rules for large-scale query plans.

B. Problem Definition

- **Objective:** Given a set of query plan templates, find a pair (T_1, T_2) that satisfies the constraints (C) , such that T_1 is logically equivalent to T_2 and T_2 is more efficient.
- **The "Large-Node" Challenge:** As the number of operator nodes (n) increases, the number of possible templates grows exponentially $(O(2^n))$, making brute-force verification computationally infeasible.
- **Input:** A set of operators O and max nodes n . A verifier V (SMT solver) for logical equivalence. A workload W of real-world queries. A ranking model S to predict rule effectiveness.
- **Output:** A set of rules $\Sigma = \{(T_1 \rightarrow T_2) | C\}$

C. Preliminary

- **Query Plan Template (T):** An abstraction of a SQL query where specific table/column names are replaced with variables, preserving only the operator structure.
- **Rule Definition:** A rewrite rule is defined as follows:

$$R(T_{src}, T_{dest}, M, C)$$

- T_{src} / T_{dest} : The source and target plan templates.
- M (Mappings): Correlations between variables in T_{src} and T_{dest} .
- C (Constraints): Requirements for the rule to be valid.
- **Rule Equivalence:** $T_{src} \equiv T_{dest}$ under M and C , verified by SMT solvers.

D. Contribution

1. Standardized Template Enumeration

- Eliminating structural heterogeneity and identifier operators through a standardized framework significantly reduces redundant search space, enabling efficient large-scale node enumeration.

2. Guided Discovery Based on Learning to Rank (LTR)

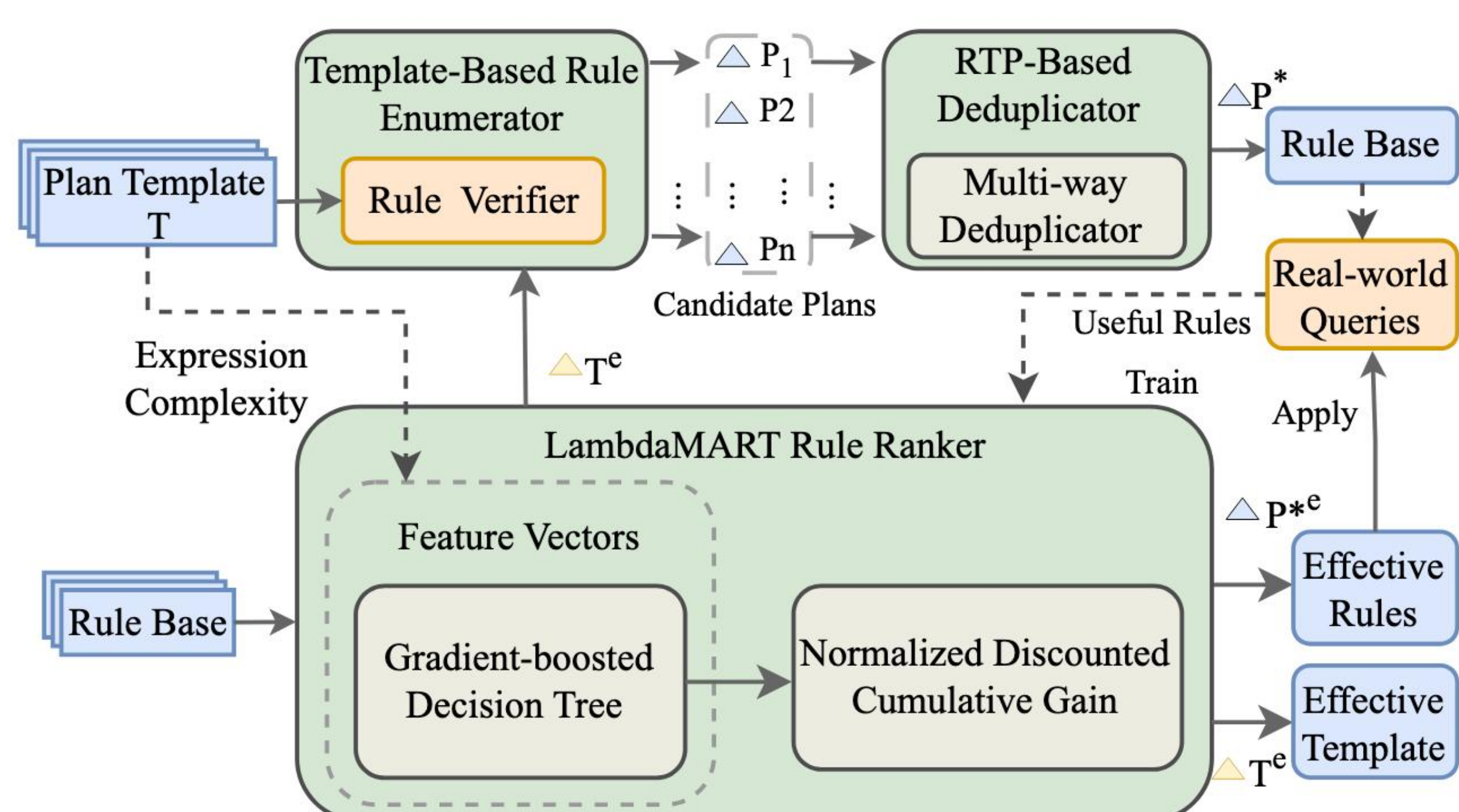
- Using the LambdaMART model to prioritize highly useful template pairs and filter out low-value candidate templates, thus bypassing exponential validation bottlenecks.

3. Large-Scale Rule Base

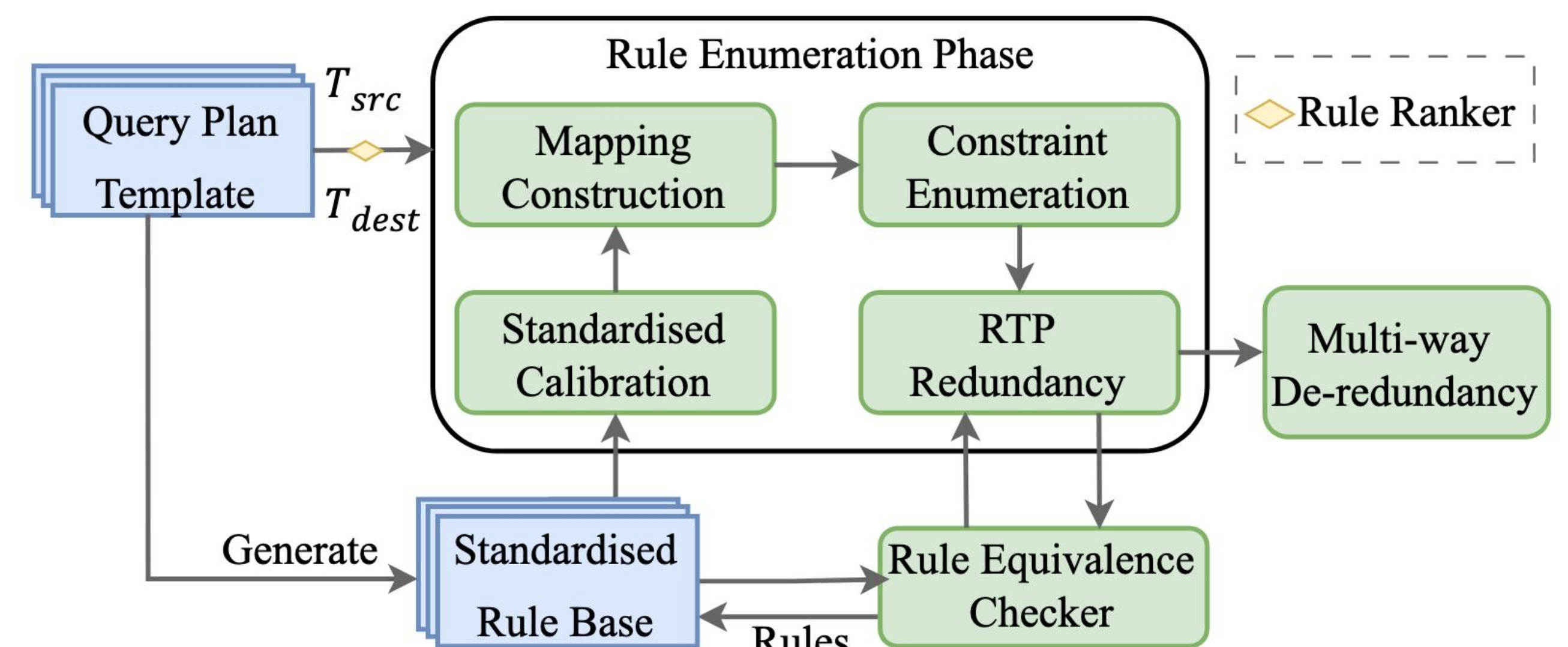
- Building the **largest rewritten rule base** to date, containing **over 1 million non-redundant and empirically validated rules**.

4. High-Performance Discovery and Practical Functionality

- Achieving up to **87% faster** runtime compared to state-of-the-art tools and identifying new, complex rule categories, significantly optimizing production-level SQL workloads.



E. Standardized Enumeration & RTP



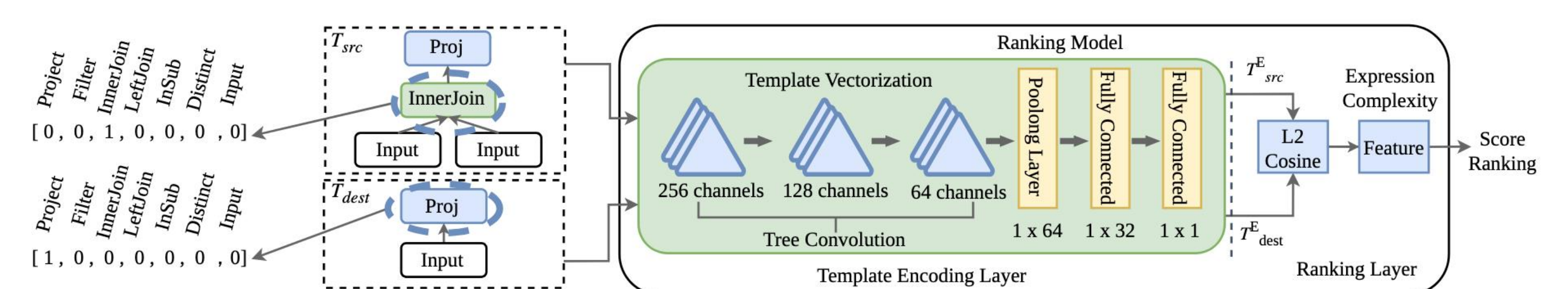
1. Eliminating Structural Redundancy: Reduces search space by converting logically identical trees into a unique Canonical Form:

- ① **Operator Removal:** Pruning identity operators that do not alter the result (e.g., Filter operators with predicates that are always true, or empty Join operators).
- ② **Operator Permutation:** For operators exhibiting commutativity or associativity, a fixed canonical order is enforced (e.g., a priority order of Project > Distinct > Filter) to consolidate isomorphic structures.

2. The RTP (Representative Template Pair) Algorithm: To further compress the search space, SLER introduces the RTP algorithm to identify and prune logically redundant rule pairs:

- ① **Logical Equivalence Pruning:** Skips template pairs (T_1, T_2) that are structural variants or logically covered by already discovered rules.
- ② **Efficient Deduplication:** Maintains a minimal set of unique candidate pairs, ensuring the SMT solver only processes non-trivial transformations.

F. Learning-to-Rank (LTR)



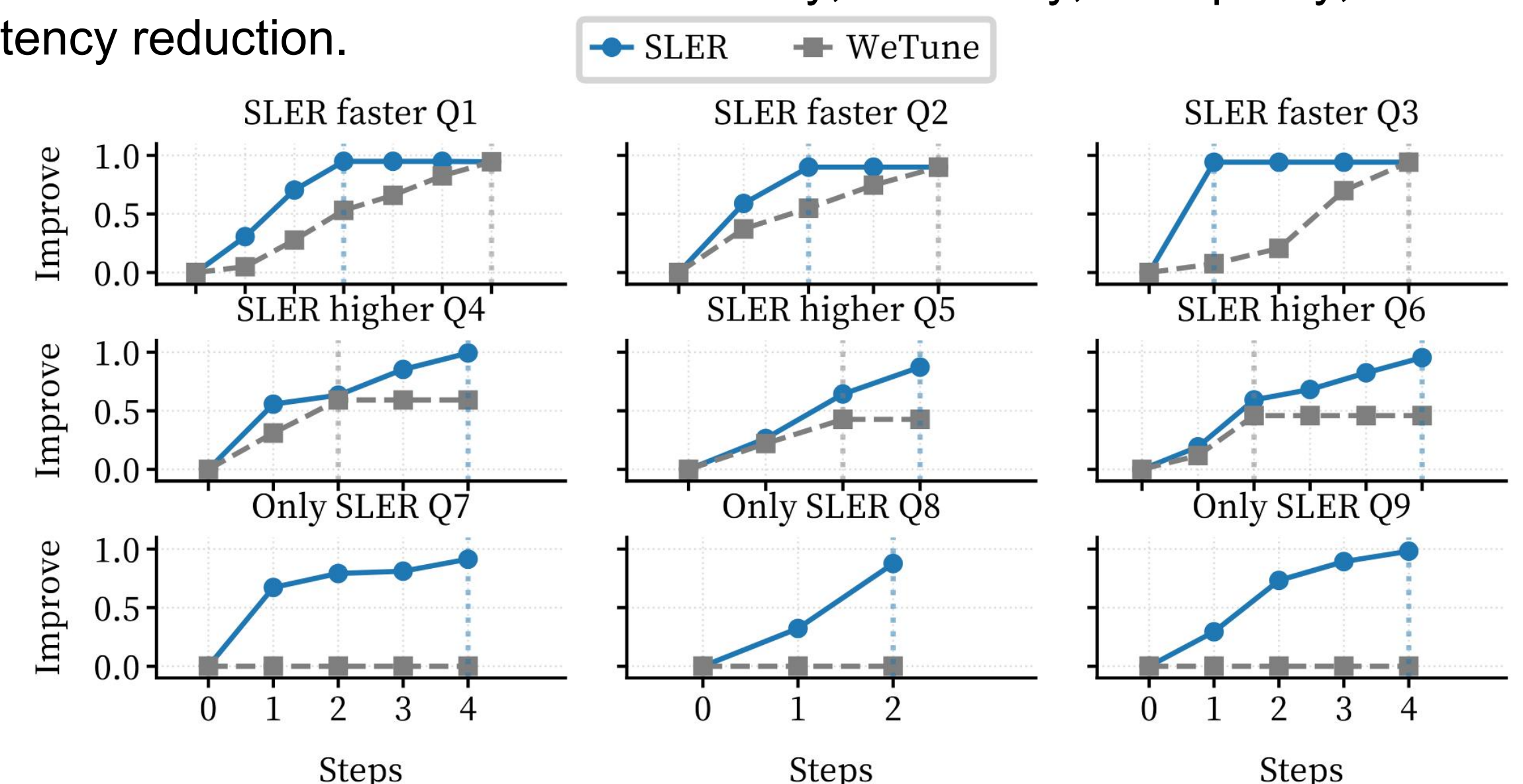
- **Approach:** Treats rule discovery as a ranking problem to prioritize "promising" template pairs.
- **Model:** LambdaMART trained on structural features of the plan trees.
- **Feature Engineering:** 9-dimensional features including tree depth, operator types, and expression complexity.

1. Pre-filtering (Discovery Phase) 2. Final Ranking (Optimization)

- **Ranks and selects Top-k promising template pairs.**
- **Enables discovery for $n \geq 5$ nodes;** slashes enumeration time while preserving high-value rules.
- **Scores and sorts validated rules by predicted performance gains.**
- **Maximizes Precision@n,** ensuring top-tier rules are prioritized for production deployment.

G. Experiments

- **Datasets.** 11,518 real-world SQL queries (GitHub Apps & YashanDB).
- **Baselines:** WeTune (State-of-the-art automated rule discovery tool).
- **Measurements.** Enumeration efficiency, scalability, rule quality, and end-to-end latency reduction.



- **Efficiency:** **34%–87%** faster than WeTune for 4-node templates; RTP pruning reduces SMT calls by **90%**.
- **Scalability breakthrough:** First to achieve **10-node discovery** (5,127 rules in 523 mins); previous SOTA timed out at $n \geq 5$.
- **Precision:** LambdaMART ensures **top-ranked rules** consistently outperform random sets.
- **Performance:** Achieves **41.8%–73.2% latency reduction**.
- **Optimization Speed:** Reaches optimal plans in **1 step** vs. WeTune's 3–4 steps.
- **Core Discovery Capabilities:** Identified **over 1 million rules**, including entirely new rules.